31.3 配置热重载

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200k tokens，输出 ~8k tokens（本节）

生产环境中，修改配置后不得不重启整个服务是一件代价高昂的事——正在进行的对话会中断，WebSocket 连接会断开，所有状态被清洗。OpenClaw 的配置热重载系统通过"文件监听 → 差异比对 → 规则路由 → 分级执行"四个阶段，让大多数配置变更可以在不中断服务的前提下即时生效。本节剖析这套机制的完整实现。

31.3.1 配置重载机制（`src/gateway/config-reload.ts`）

整体架构

配置热重载的核心流程可以概括为一条管道：

```
文件变更 → chokidar 监听 → debounce(300ms) → 读取快照 → diffConfigPaths
→ buildGatewayReloadPlan → 模式判定(off/restart/hot/hybrid) → 执行重载
```

文件监听：chokidar

OpenClaw 使用 `chokidar` 库来监视配置文件的变化。`startGatewayConfigReloader` 在启动时创建一个 watcher：

```
// src/gateway/config-reload.ts（简化）

const watcher = chokidar.watch(opts.watchPath, {
  ignoreInitial: true,                        // 不触发初始扫描事件
  awaitWriteFinish: {
    stabilityThreshold: 200,                   // 文件写入稳定后 200ms 才认为完成
    pollInterval: 50,                          // 轮询间隔 50ms
  },
  usePolling: Boolean(process.env.VITEST),    // 测试环境使用轮询模式
});

watcher.on("add", schedule);      // 新文件
watcher.on("change", schedule);   // 文件修改
watcher.on("unlink", schedule);   // 文件删除
```

> **衍生解释**： `chokidar` 是 Node.js 生态中最流行的文件监听库，它封装了各操作系统的原生文件系统事件（Linux 的 `inotify` 、macOS 的 `FSEvents` 、Windows 的 `ReadDirectoryChangesW` ），提供统一的跨平台 API。 `awaitWriteFinish` 选项解决了一个常见问题：编辑器保存文件时可能分多次写入（先清空再写入内容），如果不等待写入稳定，可能会读到一个半截的文件。

防抖（Debounce）

收到文件变更事件后，不会立即执行重载，而是通过防抖延迟 300ms：

```
// src/gateway/config-reload.ts（简化）

const DEFAULT_RELOAD_SETTINGS = {
  mode: "hybrid",
  debounceMs: 300,
};

const schedule = () => {
  if (stopped) return;
  if (debounceTimer) clearTimeout(debounceTimer);
  debounceTimer = setTimeout(() => {
    void runReload();
  }, settings.debounceMs);
};
```

> **衍生解释**：**防抖（Debounce）**是一种控制函数调用频率的技术。当事件连续触发时，只在最后一次触发后等待一段时间（此处为 300ms）才执行回调。这与节流（Throttle）**不同——节流保

> 证固定时间间隔内至少执行一次，而防抖保证"安静"一段时间后才执行。在配置热重载场景中，用户可能连续保存多次文件，防抖确保只有最后一次保存才触发重载逻辑。

差异比对：diffConfigPaths

重载流程的核心是比较"上一份配置"和"新配置"之间有哪些路径发生了变化。 `diffConfigPaths` 递归遍历两个配置对象，返回所有值不同的路径：

```
// src/gateway/config-reload.ts

export function diffConfigPaths(prev: unknown, next: unknown, prefix = ""): string[] {
  // 严格相等 → 无变化
  if (prev === next) return [];

  // 两侧都是对象 → 递归比较每个键
  if (isPlainObject(prev) && isPlainObject(next)) {
    const keys = new Set([...Object.keys(prev), ...Object.keys(next)]);
    const paths: string[] = [];
    for (const key of keys) {
      const childPrefix = prefix ? `${prefix}.${key}` : key;
      const childPaths = diffConfigPaths(prev[key], next[key], childPrefix);
      paths.push(...childPaths);
    }
    return paths;
  }

  // 两侧都是数组 → 逐元素比较
  if (Array.isArray(prev) && Array.isArray(next)) {
    if (prev.length === next.length && prev.every((val, idx) => val === next[idx])) {
      return [];
    }
  }

  // 其他情况 → 该路径有变化
  return [prefix || "<root>"];
}
```

例如，如果用户把 `cron.schedule` 从 `"*/5 * * * *"` 改成了 `"*/10 * * * *"` ， `diffConfigPaths` 会返回 `["cron.schedule"]` 。

规则路由：buildGatewayReloadPlan

配置变更路径得到后，下一步是决定"怎么重载"。OpenClaw 维护了一个有优先级的规则表——每个配置路径前缀对应三种处理方式之一：

类别	含义	执行方式
`"restart"`	必须完全重启 Gateway	发送 SIGUSR1 信号触发进程重启
`"hot"`	可以热重载	仅重启受影响的子系统
`"none"`	无需重载	忽略（下次读取时自然生效）

规则表分为三层：基础规则（头部）、渠道插件规则（动态）、基础规则（尾部）：

```
// src/gateway/config-reload.ts

// 头部规则 —— 最高优先级
const BASE_RELOAD_RULES: ReloadRule[] = [
  { prefix: "gateway.remote",             kind: "none" },
  { prefix: "gateway.reload",             kind: "none" },
  { prefix: "hooks.gmail",                kind: "hot", actions: ["restart-gmail-watcher
  { prefix: "hooks",                      kind: "hot", actions: ["reload-hooks"] },
  { prefix: "agents.defaults.heartbeat",  kind: "hot", actions: ["restart-heartbeat"] }
  { prefix: "cron",                       kind: "hot", actions: ["restart-cron"] },
  { prefix: "browser",                    kind: "hot", actions: ["restart-browser-contr
];

// 尾部规则 —— 兜底
const BASE_RELOAD_RULES_TAIL: ReloadRule[] = [
  { prefix: "identity",   kind: "none" },
  { prefix: "models",     kind: "none" },
  { prefix: "agents",     kind: "none" },
  { prefix: "tools",      kind: "none" },
  { prefix: "skills",     kind: "none" },
  // ... 其他 "none" 类
  { prefix: "plugins",    kind: "restart" },
  { prefix: "gateway",    kind: "restart" },
  { prefix: "discovery",  kind: "restart" },
  { prefix: "canvasHost", kind: "restart" },
];
```

插件规则注入：渠道插件（Channel Plugin）可以通过 `plugin.reload.configPrefixes` 和 `plugin.reload.noopPrefixes` 声明自己的重载规则。这些规则插入到头部和尾部之间：

```
function listReloadRules(): ReloadRule[] {
  const channelReloadRules = listChannelPlugins().flatMap((plugin) => [
    // configPrefixes → hot 重载，重启对应渠道
    ...(plugin.reload?.configPrefixes ?? []).map((prefix) => ({
      prefix,
      kind: "hot" as const,
      actions: [`restart-channel:${plugin.id}` as ReloadAction],
    })),
    // noopPrefixes → 无需重载
    ...(plugin.reload?.noopPrefixes ?? []).map((prefix) => ({
      prefix,
      kind: "none" as const,
    })),
  ]);

  return [...BASE_RELOAD_RULES, ...channelReloadRules, ...BASE_RELOAD_RULES_TAIL];
}
```

> **设计要点**：规则匹配采用**最先匹配（first match）**策略——`matchRule` 线性遍历规则表，返回第一个前缀匹配的规则。这意味着头部规则的优先级高于尾部。例如 `hooks.gmail` 的规则在 `hooks` 之前，所以 Gmail 钩子的变更会触发 `restart-gmail-watcher` 而非通用的 `reload-hooks` 。

构建重载计划

`buildGatewayReloadPlan` 遍历所有变更路径，根据匹配到的规则构建一个详细的重载计划：

```typescript
// src/gateway/config-reload.ts（简化）

export function buildGatewayReloadPlan(changedPaths: string[]): GatewayReloadPlan {
  const plan: GatewayReloadPlan = {
    changedPaths,
    restartGateway: false,
    restartReasons: [],
    hotReasons: [],
    reloadHooks: false,
    restartGmailWatcher: false,
    restartBrowserControl: false,
    restartCron: false,
    restartHeartbeat: false,
    restartChannels: new Set(),
    noopPaths: [],
  };

  for (const path of changedPaths) {
    const rule = matchRule(path);
    if (!rule) {
      // 未匹配任何规则 → 安全起见，标记为需要重启
      plan.restartGateway = true;
      plan.restartReasons.push(path);
      continue;
    }
    if (rule.kind === "restart") {
      plan.restartGateway = true;
      plan.restartReasons.push(path);
    } else if (rule.kind === "none") {
      plan.noopPaths.push(path);
    } else {
      // hot —— 执行对应的动作
      plan.hotReasons.push(path);
      for (const action of rule.actions ?? []) {
        applyAction(action);  // 设置对应的 boolean 标志
      }
    }
  }

  // Gmail watcher 依赖 hooks，如果重启 Gmail，也要重载 hooks
  if (plan.restartGmailWatcher) {
    plan.reloadHooks = true;
  }
```

```
  return plan;
}
```

注意一个安全性设计：如果某个变更路径未匹配到任何规则（ `rule === null` ），系统默认将其视为 `"restart"` ——宁可多重启也不遗漏关键配置变更。

四种重载模式

重载计划构建后，根据 `gateway.reload.mode` 配置决定如何执行：

模式	行为
`"off"`	完全禁用热重载，忽略一切文件变更
`"restart"`	不管变更内容，一律重启 Gateway
`"hot"`	仅执行热重载部分；需要重启的变更被忽略（日志警告）
`"hybrid"` （默认）	优先热重载；如果有需要重启的变更，则触发完整重启

```
// runReload 中的模式判定逻辑（简化）

if (settings.mode === "off") {
  // 完全禁用
  return;
}
if (settings.mode === "restart") {
  // 直接重启
  opts.onRestart(plan, nextConfig);
  return;
}
if (plan.restartGateway) {
  if (settings.mode === "hot") {
    // hot 模式下忽略需要重启的变更
    log.warn("config reload requires gateway restart; hot mode ignoring");
    return;
  }
  // hybrid 模式 → 触发重启
  opts.onRestart(plan, nextConfig);
  return;
}
// 无需重启 → 执行热重载
await opts.onHotReload(plan, nextConfig);
```

并发保护与重入防护

热重载过程中需要防止并发执行（例如用户在重载进行时又保存了文件）：

```
let pending = false;
let running = false;
let restartQueued = false;

const runReload = async () => {
  if (running) {
    pending = true;  // 标记有待处理的重载
    return;
  }
  running = true;
  try {
    // ... 执行重载逻辑 ...
  } finally {
    running = false;
    if (pending) {
      pending = false;
      schedule();  // 重新调度
    }
  }
};
```

这是一个经典的**互斥锁（mutex）+排队**模式： `running` 标志确保同一时间只有一个重载在执行； `pending` 标志确保在重载期间到达的新变更不会丢失，而是在当前重载完成后重新触发。

31.3.2 运行时覆盖（ `src/config/runtime-overrides.ts` ）

除了文件级别的配置修改，OpenClaw 还支持在运行时通过 API 临时覆盖配置值——这些覆盖存储在内存中，不会写入磁盘，进程重启后失效。

覆盖树

运行时覆盖使用一个模块级变量 `overrides` 存储，它的结构与 `OpenClawConfig` 相同，但只包含被覆盖的路径：

```
// src/config/runtime-overrides.ts

type OverrideTree = Record<string, unknown>;

let overrides: OverrideTree = {};
```

设置与取消覆盖

`setConfigOverride` 和 `unsetConfigOverride` 通过配置路径语法操作覆盖树：

```
// src/config/runtime-overrides.ts

export function setConfigOverride(
  pathRaw: string,
  value: unknown,
): { ok: boolean; error?: string } {
  const parsed = parseConfigPath(pathRaw);
  if (!parsed.ok || !parsed.path) {
    return { ok: false, error: parsed.error ?? "Invalid path." };
  }
  setConfigValueAtPath(overrides, parsed.path, value);
  return { ok: true };
}

export function unsetConfigOverride(pathRaw: string): {
  ok: boolean; removed: boolean; error?: string;
} {
  const parsed = parseConfigPath(pathRaw);
  if (!parsed.ok || !parsed.path) {
    return { ok: false, removed: false, error: parsed.error ?? "Invalid path." };
  }
  const removed = unsetConfigValueAtPath(overrides, parsed.path);
  return { ok: true, removed };
}
```

路径解析使用 `parseConfigPath` （见 23.1 节），支持点分和方括号语法（如 `agents.list[0].model.primary` ），并内置了原型链污染防护。

深度合并：覆盖如何生效

`applyConfigOverrides` 在配置加载流水线的最后一步被调用，将覆盖树深度合并到配置对象上：

```ts
// src/config/runtime-overrides.ts

function mergeOverrides(base: unknown, override: unknown): unknown {
  // 如果双方都不是对象 → 覆盖值直接替换
  if (!isPlainObject(base) || !isPlainObject(override)) {
    return override;
  }
  // 双方都是对象 → 递归合并
  const next = { ...base };
  for (const [key, value] of Object.entries(override)) {
    if (value === undefined) continue;
    next[key] = mergeOverrides(base[key], value);
  }
  return next;
}

export function applyConfigOverrides(cfg: OpenClawConfig): OpenClawConfig {
  if (!overrides || Object.keys(overrides).length === 0) {
    return cfg;  // 无覆盖 → 直接返回
  }
  return mergeOverrides(cfg, overrides) as OpenClawConfig;
}
```

合并语义如下：

基础值	覆盖值	结果
对象	对象	递归合并（保留基础值中未被覆盖的键）
对象	非对象	覆盖值替换
非对象	任意	覆盖值替换
任意	`undefined`	保持基础值

使用场景：运行时覆盖通常由 Web 控制台的"临时调试"功能触发，例如临时切换模型、临时调高日志级别等，无需修改配置文件。

31.3.3 服务器重载处理器（ src/gateway/server-reload-handlers.ts ）

热重载计划构建完成后，实际执行重载的工作由 `createGatewayReloadHandlers` 工厂函数创建的两个处理器完成： `applyHotReload` （热重载）和 `requestGatewayRestart` （完整重启）。

工厂模式与状态注入

`createGatewayReloadHandlers` 接收一组依赖和状态访问器，返回两个处理函数。这种设计将重载逻辑与 Gateway 的具体状态解耦：

```
// src/gateway/server-reload-handlers.ts（简化）

type GatewayHotReloadState = {
  hooksConfig: ReturnType<typeof resolveHooksConfig>;
  heartbeatRunner: HeartbeatRunner;
  cronState: GatewayCronState;
  browserControl: BrowserControlServer | null;
};

export function createGatewayReloadHandlers(params: {
  deps: CliDeps;
  broadcast: (event: string, payload: unknown) => void;
  getState: () => GatewayHotReloadState;
  setState: (state: GatewayHotReloadState) => void;
  startChannel: (name: ChannelKind) => Promise<void>;
  stopChannel: (name: ChannelKind) => Promise<void>;
  // ... 各子系统的日志器
}) {
  return { applyHotReload, requestGatewayRestart };
}
```

applyHotReload：分子系统重载

`applyHotReload` 根据重载计划中的标志位，逐个重启受影响的子系统：

```typescript
// src/gateway/server-reload-handlers.ts（简化）

const applyHotReload = async (plan: GatewayReloadPlan, nextConfig: OpenClawConfig) => {
  const state = params.getState();
  const nextState = { ...state };

  // 1. 钩子重载
  if (plan.reloadHooks) {
    nextState.hooksConfig = resolveHooksConfig(nextConfig);
  }

  // 2. 心跳重载
  if (plan.restartHeartbeat) {
    nextState.heartbeatRunner.updateConfig(nextConfig);
  }

  // 3. 定时任务重载（stop-then-start）
  if (plan.restartCron) {
    state.cronState.cron.stop();
    nextState.cronState = buildGatewayCronService({
      cfg: nextConfig, deps: params.deps, broadcast: params.broadcast
    });
    void nextState.cronState.cron.start().catch(/* ... */);
  }

  // 4. 浏览器控制服务重载
  if (plan.restartBrowserControl) {
    if (state.browserControl) await state.browserControl.stop();
    nextState.browserControl = await startBrowserControlServerIfEnabled();
  }

  // 5. Gmail 监听器重载
  if (plan.restartGmailWatcher) {
    await stopGmailWatcher();
    if (!isTruthyEnvValue(process.env.OPENCLAW_SKIP_GMAIL_WATCHER)) {
      const result = await startGmailWatcher(nextConfig);
      if (result.started) log.info("gmail watcher started");
    }
  }

  // 6. 渠道重载
  if (plan.restartChannels.size > 0) {
    for (const channel of plan.restartChannels) {
```

```
      await params.stopChannel(channel);
      await params.startChannel(channel);
    }
  }

  // 7. 并发度更新
  setCommandLaneConcurrency(CommandLane.Cron,     nextConfig.cron?.maxConcurrentRuns ??
  setCommandLaneConcurrency(CommandLane.Main,     resolveAgentMaxConcurrent(nextConfig)
  setCommandLaneConcurrency(CommandLane.Subagent, resolveSubagentMaxConcurrent(nextConf

  // 8. 清除目录缓存
  resetDirectoryCache();

  // 9. 提交新状态
  params.setState(nextState);
};
```

几个设计要点：

1. **stop-then-start 模式**：定时任务、浏览器控制、Gmail 监听器都采用"先停止旧实例，再启动新实例"的方式，避免资源泄漏。
2. **环境变量跳过**：`OPENCLAW_SKIP_GMAIL_WATCHER` 和 `OPENCLAW_SKIP_CHANNELS` 环境变量允许在开发/调试时跳过某些子系统的重启。
3. **并发度即时更新**：命令队列的并发度（主 Agent、子 Agent、Cron 任务各自的并发上限）在每次热重载时同步更新。
4. **目录缓存清除**：`resetDirectoryCache()` 确保 Agent 目录解析不会使用过时的缓存。

requestGatewayRestart：完整重启

当变更无法热重载时，系统通过 Unix 信号触发 Gateway 进程重启：

```ts
// src/gateway/server-reload-handlers.ts（简化）

const requestGatewayRestart = (plan: GatewayReloadPlan, nextConfig: OpenClawConfig) =>
  setGatewaySigusr1RestartPolicy({
    allowExternal: nextConfig.commands?.restart === true
  });

  const reasons = plan.restartReasons.length
    ? plan.restartReasons.join(", ")
    : plan.changedPaths.join(", ");
  log.warn(`config change requires gateway restart (${reasons})`);

  // 检查是否有 SIGUSR1 监听器
  if (process.listenerCount("SIGUSR1") === 0) {
    log.warn("no SIGUSR1 listener found; restart skipped");
    return;
  }

  authorizeGatewaySigusr1Restart();
  process.emit("SIGUSR1");
};
```

> **衍生解释**： `SIGUSR1` 是 POSIX 标准定义的用户自定义信号（User-defined Signal 1）。在 Node.js 中， `process.emit("SIGUSR1")` 不会终止进程，而是触发注册的监听器。OpenClaw Gateway 在启动时注册了一个 SIGUSR1 监听器来执行优雅重启（graceful restart）——先停止接受新请求，等待进行中的请求完成，然后重新初始化所有子系统。 `authorizeGatewaySigusr1Restart()` 设置一个授权标志，防止外部进程的 SIGUSR1 信号意外触发重启。

热重载的完整数据流

将三个子节的内容综合起来，整个配置热重载的数据流如下：

```
用户编辑 openclaw.json
          │
    chokidar 检测到文件变更
          │
    debounce 300ms
          │
    readSnapshot() ─── 读取并验证新配置
          │                    │
          │          配置无效?  → 跳过（日志警告）
          │
    diffConfigPaths(oldConfig, newConfig)
          │                    │
          │          无变化?  → 跳过
          │
    buildGatewayReloadPlan(changedPaths)
          │
          │
    ┌─────────────────────────────────┐
    │ 模式判定                         │
    ├─────────────────────────────────┤
    │ off     → 跳过                   │
    │ restart → requestGatewayRestart  │
    │ hot     → applyHotReload         │
    │          （需重启的变更被忽略）    │
    │ hybrid  → 需重启?                │
    │           ├ 是 → requestRestart  │
    │           └ 否 → applyHotReload  │
    └─────────────────────────────────┘
          │
    applyHotReload
          │
          │
    ┌─────────────────────────────────┐
    │ 按子系统分别重载:                 │
    │  • reloadHooks → resolveHooksConfig │
    │  • restartHeartbeat → updateConfig  │
    │  • restartCron → stop + rebuild     │
    │  • restartBrowser → stop + restart  │
    │  • restartGmail → stop + restart    │
    │  • restartChannels → stop + start   │
    │  • 更新并发度                       │
    │  • 清除目录缓存                     │
    └─────────────────────────────────┘
```

本节小结

1. **文件监听**使用 chokidar，配合 `awaitWriteFinish` 等待文件写入稳定，避免读取不完整的配置。
2. **防抖机制**（默认 300ms）避免连续保存触发多次重载，同时互斥锁+排队保证不会并发执行重载。
3. **差异比对**（`diffConfigPaths`）递归比较新旧配置，返回所有变更的路径列表。
4. **规则路由**将每个变更路径映射到 `restart` / `hot` / `none` 三种处理方式，规则表支持渠道插件动态注入。
5. **四种重载模式**——`off`（禁用）、`restart`（总是重启）、`hot`（仅热重载）、`hybrid`（混合，默认）——提供了灵活的策略选择。
6. **运行时覆盖**提供纯内存的配置覆盖机制，通过深度合并叠加在磁盘配置之上，适用于临时调试。
7. **热重载处理器**按子系统分别执行 stop-then-start，确保每个子系统独立重载且不泄漏资源。
8. **完整重启**通过 SIGUSR1 信号触发优雅重启，带有授权机制防止意外触发。